

CS Fundamentals - DBMS, OS, CN, CD

The core-subject question bank

Nirmalya Mandal - SAP Labs Interview Prep

Study pack - part 6 of 8

Contents

How to use this	3
DBMS - Database Management Systems	4
Operating Systems (OS)	7
Computer Networks (CN)	9
Compiler Design (CD)	12
The revision priority	14

How to use this

Your seniors said it directly: “**fundamentals of DBMS and Data Structures are most important,**” and the technical round mixes core CS subjects. This document covers the four that come up most:

- **DBMS** (most important - study this first),
- **Operating Systems (OS)**,
- **Computer Networks (CN)**,
- **Compiler Design (CD)** - which you can speak to naturally because your CONTEXT interpreter project is literally compiler design.

Each item is a likely question with a short, correct answer **at your level**. Keep answers conversational, and wherever you can, **tie it back to a project** - it proves you didn't just memorise a definition. The tie-in prompts are marked (*link:*).

DBMS - Database Management Systems

The most-tested subject. You use PostgreSQL daily, so speak from experience.

What is a DBMS? DBMS vs RDBMS?

A **DBMS** is software to store, manage, and query data. An **RDBMS** (Relational DBMS) is a DBMS that stores data in **tables (relations)** with rows and columns and supports relationships between them using keys - PostgreSQL, MySQL, Oracle. (*link: I use PostgreSQL, an RDBMS, across my projects.*)

What are the ACID properties?

The four guarantees a **transaction** provides:

- **Atomicity** - all steps happen or none do.
- **Consistency** - the database stays in a valid state, never breaking its rules.
- **Isolation** - concurrent transactions don't interfere; the result is as if they ran one by one.
- **Durability** - once committed, data survives crashes.

(*link: DokLink's bed booking relies on Atomicity and Isolation - that's how I stopped two users booking the last ICU bed at once.*)

What is a transaction?

A group of database operations treated as a **single unit of work** that either fully succeeds (commit) or fully undoes (rollback). It's the mechanism that gives you ACID.

Explain keys: primary, foreign, candidate, super, composite.

- **Super key** - any set of columns that uniquely identifies a row.
- **Candidate key** - a minimal super key (no extra columns).
- **Primary key** - the candidate key you choose as the main unique identifier; cannot be null.
- **Foreign key** - a column that references another table's primary key, creating a relationship.
- **Composite key** - a primary key made of two or more columns together.

What is normalization? Explain the normal forms.

Organising tables to **reduce redundancy** and avoid update problems, by splitting data into related tables.

- **1NF**: atomic values, no repeating groups (each cell holds one value).
- **2NF**: 1NF + every non-key column depends on the **whole** primary key (removes partial dependency).
- **3NF**: 2NF + no non-key column depends on another non-key column (removes transitive dependency).
- **BCNF**: a stricter 3NF where every determinant is a candidate key.

What is denormalization and why do it?

Deliberately adding some redundancy (merging tables, duplicating a column) to make **reads faster** by avoiding joins. The trade-off is more storage and harder updates. Used when read performance matters more than perfect normalization.

What is a JOIN? Name the types.

Combining rows from two tables based on a related column.

- **INNER JOIN** - only rows matching in both tables.
- **LEFT JOIN** - all rows from the left table, plus matches from the right (nulls where none).
- **RIGHT JOIN** - all from the right, plus matches from the left.
- **FULL OUTER JOIN** - all rows from both, matched where possible.
- **CROSS JOIN** - every combination (Cartesian product).

What is an index? What is the trade-off?

A data structure (usually a **B-tree**) that makes lookups fast by avoiding a full table scan, like a book's index. The trade-off: it uses extra storage and slightly slows down inserts/updates (the index must be maintained). *(link: I used indexed geo-lookups so DokLink finds nearby hospitals quickly.)*

DELETE vs TRUNCATE vs DROP?

- **DELETE** - removes specific rows (can use WHERE); can be rolled back; slower.
- **TRUNCATE** - removes all rows fast; can't use WHERE; resets the table.
- **DROP** - deletes the whole table (structure and data).

DDL, DML, DCL, TCL - what are these?

Categories of SQL commands:

- **DDL** (Data Definition) - CREATE, ALTER, DROP (structure).
- **DML** (Data Manipulation) - SELECT, INSERT, UPDATE, DELETE (data).
- **DCL** (Data Control) - GRANT, REVOKE (permissions).
- **TCL** (Transaction Control) - COMMIT, ROLLBACK, SAVEPOINT.

SQL vs NoSQL? When would you choose each?

- **SQL (PostgreSQL):** structured tables, fixed schema, strong consistency, transactions. Best when data is relational and correctness matters.
- **NoSQL (MongoDB):** flexible documents, easy horizontal scaling, good for unstructured or fast-changing data.

(link: I chose PostgreSQL for DokLink because bookings need transactions and integrity, not flexibility.)

What is a deadlock in a database? (concurrency)

Two transactions each holding a lock the other needs, so both wait forever. Handled by detection and rolling one back, or by lock ordering. (Same idea as OS deadlock, below.)

What are isolation levels?

How much one transaction can see of another's uncommitted work - from **Read Uncommitted** (least isolated), **Read Committed**, **Repeatable Read**, to **Serializable** (most isolated, as if run one at a time). Higher isolation prevents more anomalies (dirty reads, non-repeatable reads, phantoms) but costs performance.

What is a view? A stored procedure? A trigger?

- **View** - a saved query that acts like a virtual table.
- **Stored procedure** - a saved block of SQL logic you can call by name.
- **Trigger** - code that runs automatically on an event (e.g. before an insert).

What is the ER model?

Entity-Relationship model - a design diagram of **entities** (tables/things), their **attributes** (columns), and the **relationships** between them (one-to-one, one-to-many, many-to-many). It's how you plan a schema before building it. (*link: this is exactly the scenario-DB-design method in part 4.*)

Operating Systems (OS)

What is an operating system?

Software that manages the computer's hardware and resources (CPU, memory, storage, devices) and provides services to programs. It sits between applications and hardware.

Process vs thread?

- A **process** is a running program with its own memory space.
- A **thread** is a lighter unit of execution **inside** a process; threads of one process **share** its memory.
- Threads are cheaper to create and switch between; processes are isolated and safer.

What are the states of a process?

New -> Ready -> Running -> Waiting (blocked) -> Terminated. A process cycles between Ready, Running, and Waiting as the scheduler and I/O dictate.

What is CPU scheduling? Name algorithms.

Deciding which ready process runs next.

- **FCFS** (First Come First Served) - in arrival order; simple but a long job blocks others (convoy effect).
- **SJF** (Shortest Job First) - shortest next job first; optimal average wait but needs to know job length.
- **Round Robin** - each process gets a fixed time slice; fair, good for time-sharing.
- **Priority scheduling** - highest priority first; risk of starvation for low-priority jobs.

What is a deadlock? The four conditions?

Processes each waiting for a resource another holds, so none proceed. Needs **all four** (Coffman conditions):

1. **Mutual exclusion** - a resource can't be shared.
2. **Hold and wait** - hold one resource while waiting for another.
3. **No preemption** - resources can't be forcibly taken.
4. **Circular wait** - a cycle of processes each waiting on the next.

Break any one to prevent deadlock. **Avoidance** uses the Banker's algorithm. (*link: a database deadlock is the same idea.*)

Mutex vs semaphore?

Both control access to shared resources.

- **Mutex** - a lock owned by one thread at a time (binary, one owner).
- **Semaphore** - a counter allowing up to N threads; a **binary semaphore** is like a mutex, a **counting semaphore** allows several.

What is a critical section / race condition?

A **critical section** is code that accesses shared data and must not run in two threads at once. A **race condition** is when the result depends on timing because that protection is missing. (*link: DokLink's double-booking was exactly a race condition, solved with atomic transactions and locking.*)

What is virtual memory?

An illusion that each process has a large, continuous memory, backed by RAM plus disk (swap). It lets programs use more memory than physically exists and keeps processes isolated.

Paging vs segmentation?

- **Paging** - memory split into fixed-size **pages**; avoids external fragmentation.
- **Segmentation** - memory split into variable-size **segments** by logical unit (code, stack, data).

What is a page fault? Page replacement algorithms?

A **page fault** happens when a needed page isn't in RAM and must be fetched from disk. When RAM is full, a **page replacement algorithm** decides what to evict:

- **FIFO** - evict the oldest page.
- **LRU** (Least Recently Used) - evict the one unused longest.
- **Optimal** - evict the one not needed for the longest future time (theoretical benchmark).

What is thrashing?

When the system spends more time swapping pages in and out than doing real work, because processes don't have enough RAM. Fixed by reducing the degree of multiprogramming or adding memory.

Internal vs external fragmentation?

- **Internal** - wasted space **inside** an allocated block (allocated more than needed).
- **External** - free memory exists but is **scattered** in small pieces, so a large request can't be met.

What is a context switch?

Saving the state of one process/thread and loading another's, so the CPU can switch between them. It has overhead, which is why too many switches hurt performance.

What is IPC?

Inter-Process Communication - ways processes exchange data: pipes, message queues, shared memory, sockets.

Computer Networks (CN)

What is the OSI model? The 7 layers?

A conceptual model of how data moves across a network, in 7 layers (top to bottom):

1. **Application** - user-facing protocols (HTTP, FTP).
2. **Presentation** - formatting, encryption.
3. **Session** - managing connections/sessions.
4. **Transport** - reliable delivery (TCP, UDP).
5. **Network** - routing and addressing (IP).
6. **Data Link** - node-to-node delivery, MAC addresses.
7. **Physical** - the actual cables/signals.

Mnemonic (top-down): **All People Seem To Need Data Processing**.

OSI vs TCP/IP model?

The **TCP/IP model** is the practical 4-layer version actually used on the internet: **Application, Transport, Internet, Network Access**. OSI is the 7-layer teaching model; TCP/IP is what runs the real internet.

TCP vs UDP?

- **TCP** (Transmission Control Protocol) - connection-based, **reliable**, ordered, error-checked. Used for web, email, file transfer.
- **UDP** (User Datagram Protocol) - connectionless, **fast but unreliable**, no ordering guarantee. Used for video calls, streaming, games where speed beats perfection.

Explain the TCP 3-way handshake.

How a TCP connection is established:

1. Client sends **SYN**.
2. Server replies **SYN-ACK**.
3. Client sends **ACK**.

Now both sides are synchronised and can exchange data.

What happens when you type a URL and press enter?

A classic question. Short version:

1. **DNS** resolves the domain name to an IP address.
2. A **TCP connection** is opened to that IP (3-way handshake), with **TLS** if HTTPS.
3. The browser sends an **HTTP request**.
4. The server responds with the **HTML**, and the browser fetches CSS/JS/images and renders the page.

(link: I set up DNS, HTTPS/SSL, and the Nginx server for my deployed sites, so I've configured these pieces myself.)

What is HTTP vs HTTPS?

HTTP is the protocol for fetching web pages; **HTTPS** is HTTP over **TLS encryption**, so traffic is private and tamper-proof. *(link: I automated SSL certificates with Certbot on my VPS deployments.)*

Common HTTP methods and status codes?

- **Methods:** GET (read), POST (create), PUT/PATCH (update), DELETE (remove).
- **Status codes:** 2xx success (200 OK, 201 Created), 3xx redirect (301), 4xx client error (400 Bad Request, 401 Unauthorized, 404 Not Found), 5xx server error (500). *(link: I designed REST APIs using exactly these.)*

What is DNS?

Domain Name System - the internet's phone book; it translates human names like `devniru.in` into IP addresses computers use.

What is an IP address? IPv4 vs IPv6?

A unique address identifying a device on a network. **IPv4** is 32-bit (e.g. `192.168.1.1`, about 4 billion addresses); **IPv6** is 128-bit, created because IPv4 addresses ran out.

MAC address vs IP address?

- **MAC** - a permanent hardware address of a network card (data-link layer).
- **IP** - a logical, changeable network address (network layer). MAC is local delivery; IP is end-to-end routing.

Router vs switch vs hub?

- **Hub** - dumbly broadcasts to all ports (obsolete).
- **Switch** - forwards data only to the correct device on a local network (uses MAC).
- **Router** - connects different networks and routes traffic between them (uses IP).

What is DHCP? What is ARP?

- **DHCP** (Dynamic Host Configuration Protocol) - automatically assigns IP addresses to devices on a network.
- **ARP** (Address Resolution Protocol) - maps an IP address to a MAC address for local delivery.

Flow control vs congestion control?

- **Flow control** - stops a fast sender from overwhelming a slow receiver.
- **Congestion control** - stops too much traffic from overwhelming the **network** itself.

What is a firewall?

A security system that filters incoming/outgoing network traffic based on rules, to block unwanted access.

Compiler Design (CD)

You have an edge here: your **CONTEXT interpreter** project is compiler design in practice. Speak from it.

Compiler vs interpreter?

- A **compiler** translates the whole program into machine code **ahead of time**, then it runs (C++, Java to bytecode).
- An **interpreter** reads and executes the program **directly, line by line** (Python).
- *(link: my CONTEXT project is an interpreter - it reads code and executes it directly.)*

What are the phases of a compiler?

The pipeline from source code to machine code:

1. **Lexical analysis** - break source into **tokens**.
2. **Syntax analysis (parsing)** - build a **parse tree** / **AST**, checking grammar.
3. **Semantic analysis** - check meaning (types, declared variables).
4. **Intermediate code generation** - a machine-independent middle representation.
5. **Code optimization** - make it faster/smaller.
6. **Code generation** - produce target machine code.

A **symbol table** and **error handler** support all phases. *(link: I built the first stages myself - lexer, parser, AST, and execution.)*

What is lexical analysis? Token, lexeme, pattern?

The first phase, done by the **lexer**, which breaks raw text into **tokens** (the smallest meaningful units).

- **Token** - a category, like **NUMBER** or **PLUS**.
- **Lexeme** - the actual text matched, like **42** or **+**.
- **Pattern** - the rule (often a regex) that defines a token.

(link: my lexer turns 3 + 4 into the tokens NUMBER, PLUS, NUMBER.)

What is parsing? Parse tree vs AST?

Parsing checks the tokens follow the language's grammar and builds a tree.

- A **parse tree** shows every grammar rule applied (verbose).
- An **AST** (Abstract Syntax Tree) is a cleaner tree keeping only the meaningful structure, respecting precedence.

(link: my parser builds an AST, then the interpreter walks it to compute results.)

Top-down vs bottom-up parsing?

- **Top-down** - builds the tree from the root down, predicting rules (e.g. LL parsers, recursive descent).
- **Bottom-up** - builds from the leaves up, reducing tokens into rules (e.g. LR parsers).

(link: my interpreter uses a recursive-descent, top-down style.)

What is a grammar / CFG?

A **Context-Free Grammar** is a set of rules defining valid programs in a language - productions like `expr -> expr + term`. It's the formal specification a parser follows.

What is a symbol table?

A data structure the compiler uses to store information about identifiers - variable names, their types, and scope - looked up across phases. *(link: my interpreter keeps variable bindings, which is a simple symbol table.)*

What is left recursion and why remove it?

A grammar rule that refers to itself on the left, like `A -> A + b`. It makes a top-down (recursive-descent) parser loop forever, so it must be rewritten before top-down parsing.

The revision priority

If you're short on time, revise in this order (matches what SAP asks most):

1. **DBMS** - ACID, keys, normalization, joins, indexing, SQL vs NoSQL, and the scenario-design method (part 4). This is the highest-value subject.
2. **OS** - process vs thread, deadlock (4 conditions), scheduling, virtual memory/paging, race condition.
3. **CN** - OSI/TCP-IP layers, TCP vs UDP, the 3-way handshake, "what happens when you type a URL," HTTP/HTTPS.
4. **CD** - compiler phases, compiler vs interpreter, lexer/parser/AST - all of which you can anchor to your CONTEXT project.

The winning move across all four: end each answer by linking it to something you actually built. "ACID - I relied on that in DokLink." "Race condition - that's the double-booking bug I fixed." "Compiler phases - I built a lexer and parser in my CONTEXT project." That turns a memorised definition into lived experience, which is exactly what the seniors said gets you selected.